SIMATS ENGINEERING

DSA0114 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs.

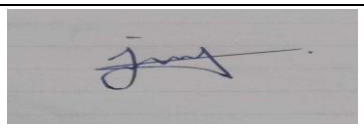
Student Name	Jannathul Firdaus. N	**Register Number**	192573025
Department / Section	B.Tech CSE - DS	**Academic Year / Semester**	2nd year
Faculty Name	Amirtha	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	07.11.2026	**Date of Evaluation**	
GitHub Repository	https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment1	**Compiler**	Code::Blocks IDE with the GNU GCC (G++) compiler.

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		

Task	Task Title	Max	Marks Awarded	Remarks
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
	TOTAL	50		Percentage:

OVERALL REMARKS

		
Student Signature	**Faculty Signature**	**Program Director**

ASSIGNMENT 1

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Assignment Title: Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System (Total Marks - 50)

Nature of Assignment: Individual

Duration: 1 Week

Submission Date: 07.11.2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- Every student must individually execute every task in this assignment, including all "Complete the Code" programs.
- Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C++ source files, input files and output screenshots.
- Include a README.md with your name, register number, compiler and version (for example g++ 13), the exact g++ command used to compile (for example g++ -std=c++17 -Wall -Wextra), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. An assignment submitted without a working GitHub link will not be evaluated.

GitHub Repository Link:
https://github.com/jannathulfirdaus17/DSA0114_192573025_Assignment1

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output with the expected output.

Predict the Output: Write the exact output of the given program before running it. Then run it and explain every difference between your prediction and the actual output, using the C++ rules involved. Predictions written after running the program will not be credited.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Namma Metro Smart-Card Fare System

A city metro rail divides its network into three zones – Inner, Middle and Outer. The fare is Rs 10, Rs 20 or Rs 30 according to the farthest zone travelled, with a 20% surcharge during peak hours (8–10
AM and 5–8 PM). Passengers use a stored-value smart card, and a journey is allowed only if at least Rs 50 remains on the card after the fare is deducted. Students and senior citizens receive a concession.

The first version of the gate software was written in a hurry. Some fares are off by a paisa, the peak check accepts 10 AM as peak and 8 PM as off-peak inconsistently, a negative balance once appeared on a card, and the fare values are typed in several places.

The number of journeys processed may grow:

1 station (20,000 journeys a day) → 1 line (500,000 a day) → whole network (3,000,000 a day)

You are required to analyse the problem, identify its objects, choose suitable C++ data types, constants, operators and control structures, implement the fare logic and validate it, so that every fare and balance is exact.

Design Challenge:

The metro is adding a fourth zone, a weekend discount and monthly passes.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Understand the Problem and Identify the Objects 4 Marks

Bloom’s Level: K5 – Evaluate

Analyse the fare problem from the object-oriented point of view.

Discuss:

- The objects in the problem (card, passenger, journey, zone, fare table) and the data and behaviour of each
- How the same problem would be written in a procedural style, and what changes in an object-oriented style
- The basic principles of OOP – encapsulation, abstraction, inheritance and polymorphism – and where each could appear in this system
- The rules every correct journey must satisfy (for example: balance never below Rs 50 after a journey)

You are expected to justify each object and rule, not merely list names.

Alternative Concept and Complexity Analysis

If the fare system were written in a purely procedural style (global variables and free functions) instead of SmartCard, Journey and FareTable classes, what would the solution be? Analyse whether the time per journey or the memory per card changes (both are O(1)), and compare how much code must be changed when one fare rule changes.

Evidence Expected

A clear object-oriented analysis with objects, their data and behaviour, rules and stated assumptions.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly 3 Marks

Bloom's Level: K5 – Evaluate

Define the fares, surcharge and limits so that a revision is made in one place only.

Discuss:

- The five kinds of tokens in your program (keywords, identifiers, constants, operators, punctuators) with examples
- #define, const and constexpr constants, and why constexpr is preferred for the fare table
- enum and enum class for zones, and why enum class is safer
- Naming rules and conventions for identifiers

Alternative Concept and Complexity Analysis

If the fares were written with #define macros instead of constexpr constants and an enum class, what would change? Compare type checking and scope, and analyse the memory used (a macro occupies no storage; a constexpr value usually does not either) and whether the running time is affected.

Evidence Expected

A single, well-named set of constants with a justified choice between #define, const, constexpr and enum class.

TASK 3 – Choose Data Types and Check Type Compatibility 4 Marks

Bloom's Level: K5 – Evaluate

Select a C++ data type for every value in the system.

Discuss:

- The type chosen (bool, char, int, long long, unsigned, float, double, auto) for fares, balances, times, zone numbers and journey counts, with sizeof on your system
- Why money should be stored in integer paise and not as double rupees
- Type compatibility rules, and what the compiler does with int, double and bool values in the same expression
- The ranges from <climits> and whether int is enough for a day's collection across the network

Alternative Concept and Complexity Analysis

If money were stored as double rupees instead of long long paise, what would the solution be? Analyse the time per addition (O(1) in both), the space per value (8 bytes in both), and measure how the rounding error grows when n fares are added.

Evidence Expected

A justified data-type table with sizes and ranges, and evidence of one precision or overflow problem with its resolution.

TASK 4 – Control Implicit Conversions 4 Marks

Bloom's Level: K5 – Evaluate

Identify every place in the program where C++ converts a value from one type to another without being told.

Discuss:

- Integer division and integer-to-double conversion
- Comparison of signed and unsigned values, and the compiler warning it produces
- Narrowing conversions, and how brace initialisation (int x{3.7};) prevents them • static_cast

and when an explicit cast is needed

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>


int main()
{
   int a = 7, b = 2;
double c = a / b;
double d =
static_cast<double>(a)
/ b;    std::cout << c <<
' ' << d << '\n';

   char ch = 'A' + 2;      std::cout <<
ch << ' ' << ch + 1 << '\n';

   bool flag = 5;        std::cout << flag << ' ' <<
std::boolalpha << flag << '\n';

   unsigned int u = 1;
   int s = -1;     std::cout << (s < u ? "s < u"
: "s >= u") << '\n';
}
```

My predicted output (write before running):

Explain each line of the actual output, naming the conversion that took place (integer division, char to int promotion, int to bool, signed to unsigned).

Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly with static_cast (and brace initialisation were used everywhere), instead of relying on implicit conversions, what would the solution be? Analyse whether the running time or memory changes, and which errors are caught at compile time instead of at run time.

Evidence Expected

Identified conversion points with demonstrated surprises, and corrected code using brace initialisation or static_cast.

TASK 5 – Write Correct Expressions with the Right Precedence 4 Marks

Bloom's Level: K5 – Evaluate

Write the expressions for the peak check, the surcharge and the concession.

Discuss:

- Operator precedence and associativity in each expression, including the conditional operator ?: and compound assignment
- The difference between x++ and ++x, and why i = i++ is a mistake
- The result of / and % with negative operands
- Why (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20) is correct and hour >= 8 && hour < 10 || hour >= 17 && hour < 20 is equivalent but harder to read

Predict the Output

Write the exact output of the program below BEFORE running it. Then compile and run it, and explain every difference between your prediction and the actual output.

```
#include <iostream>

int main()
{    int x = 5, y = 3;    std::cout << x + y * 2
<< ' ' << (x + y) * 2 << '\n';
```

```
   int z = x++ + ++y;      std::cout << z
<< ' ' << x << ' ' << y << '\n';

   std::cout << (x > y ? x : y) << ' ' << x % 4 * 2 << '\n';     std::cout <<
10 / 3 << ' ' << 10 % 3 << ' ' << -10 / 3 << ' ' << -10 % 3 << '\n';

   int k = 0;      for (int i = 0; i <
10; i += 3) k += i;     std::cout <<
k << '\n';
}
```

My predicted output (write before running):

For every line, name the operators involved and the order in which precedence and associativity apply them.

Alternative Concept and Complexity Analysis

If the peak-hour check were done with a lookup table bool peak[24] indexed by the hour, instead of a combined && / || expression, what would the solution be? Analyse the time per check (O(1) in both, but fewer comparisons with the table) and the extra space of 24 bytes.

Evidence Expected

Correct, readable expressions with an operator-by-operator explanation.

TASK 6 – Design the Control Structures 4 Marks

Bloom's Level: K5 – Evaluate

Design the decision and looping logic for computing a fare and processing a day's journeys.

Discuss:

- When to use if–else, a switch on the zone, and the conditional operator

- How a for loop, a range-based for loop and a while loop could process the journeys, and which is best
- Where break and continue are needed (for example, skipping an invalid journey) •

  What happens when no case of a switch matches

Write pseudocode for the fare computation following the SIMATS Engineering – Standard for Algorithmic Pseudocode before writing C++ code.

Complete the Code

Complete TODO 1 and TODO 2 below using a switch statement and integer paise, and execute the program.

```
#include <iomanip>

#include <iostream>


enum class Zone { Inner, Middle, Outer };

constexpr int BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, Rs 20, Rs 30

constexpr int PEAK_SURCHARGE_PERCENT = 20; constexpr long long

MIN_BALANCE_PAISE = 5000;     // Rs 50 must remain on the card


long long fare(Zone z, bool peak)

{

   // TODO 1: Use a switch on z to pick the base fare from BASE_PAISE, then add

   //   PEAK_SURCHARGE_PERCENT % in peak hours using integer arithmetic only


}


bool canTravel(long long balance, long long f)

{

   // TODO 2: Return true only if the balance left after paying the fare is at least

   //   MIN_BALANCE_PAISE
```

```
}

int main()
{
   long long balance = 10000;                    // Rs 100.00 in paise
struct Trip { Zone zone; bool peak; };
   Trip trips[] = {{Zone::Inner, false}, {Zone::Outer, true},
{Zone::Middle, false}};     for (const Trip& t : trips) {         long long f =
fare(t.zone, t.peak);         if (canTravel(balance, f)) {             balance -= f;
          std::cout << "Fare Rs " << f / 100 << '.' << std::setw(2)
                    << std::setfill('0') << f % 100 << std::setfill(' ');
       } else {
          std::cout << "Denied (low balance)";
       }
       std::cout << "  balance Rs " << balance / 100 << '\n';
   }
}
```

Expected output:

```
Fare Rs 10.00  balance Rs 90
Fare Rs 36.00  balance Rs 54
Denied (low balance)  balance Rs 54
```

Add a journey at exactly 10 AM and at exactly 8 PM and check that your peak rule treats them correctly.

Alternative Concept and Complexity Analysis

If the base fare were looked up from an array indexed by the zone (constexpr long long FARE[] =
{1000, 2000, 3000}) instead of a switch, what would the solution be? Analyse the time per lookup (O(1) in both; the switch may become a jump table), the space of the array, and how each design copes with a fourth zone.

Evidence Expected

Pseudocode and C++ control structures for the fare computation, with justified choices.

TASK 7 – From Procedural to Object-Oriented 4 Marks

Bloom's Level: K5 – Evaluate

Rewrite the fare logic in two

styles.

Discuss:

- A procedural version with separate variables and free functions
- An object-oriented version with a SmartCard class that keeps its balance private and offers deduct() and canTravel()
- How the class prevents a negative balance
- Which version is easier to change when the minimum balance changes

Alternative Concept and Complexity Analysis

If the balance were kept in a public data member instead of a private member with deduct() and canTravel(), what would the solution be? Analyse whether time or space changes (they do not), and explain which rule of the system can then be broken by any part of the program.

Evidence Expected

Two working versions and a reasoned comparison of the procedural and object-oriented designs.

TASK 8 – Read and Print Data Safely 4 Marks

Bloom's Level: K5 – Evaluate

Design the input and output of the gate program.

Discuss:

- Reading values with cin and checking cin.fail() when the operator types letters instead of numbers
- Clearing the error state with cin.clear() and discarding the bad input
- Formatting money with <iomanip> (setw, setfill, fixed, setprecision)
- Printing a neat journey report

Alternative Concept and Complexity Analysis

If each input line were read with std::getline() and converted with std::stoi() inside try/catch, instead of reading numbers directly with cin >>, what would the solution be? Analyse the time to process a line of length L (O(L) in both) and the buffer space needed, and compare how each recovers from invalid input.

Evidence Expected

Safe input handling with demonstrated recovery from invalid input, and correctly formatted output.

TASK 9 – Implement and Test the Fare Logic 4 Marks

Bloom's Level: K5 – Evaluate

Implement the complete fare logic and test it.

Test:

C. Normal journeys in each zone, peak and off-peak

D. Boundary cases: exactly 8:00, 10:00, 17:00 and 20:00; a balance that leaves exactly Rs 50

E. Invalid cases: an unknown zone, a negative recharge, letters typed for the hour • A small set of 5 journeys whose total you calculate by hand

Alternative Concept and Complexity Analysis

If each journey were tested against a table of test cases (inputs and expected fares) in a loop, instead of separate hand-written checks, what would the solution be? Analyse the time for t tests (O(t)) and the space of the table, and how easily a new boundary case can be added.

Evidence Expected

Working source code and a test table with expected and actual results for every case.

TASK 10 – Compare Expectations with Execution 4 Marks

Bloom's Level: K5 – Evaluate

Compare what you expected with what actually happened.

Discuss:

F. Whether the double and integer-paise totals for 20,000, 500,000 and 3,000,000 journeys agree, and by how much they differ

G. Whether int overflows for the network total

H. The compiler warnings you received with -Wall -Wextra and what each meant • Reasons for any difference

Alternative Concept and Complexity Analysis

If the network total were accumulated in an int, a long long or a double, what would happen at 3,000,000 journeys? Analyse when each type fails (overflow versus loss of precision), and compare the time per addition and the space per variable.

Evidence Expected

A reasoned comparison of expected and observed behaviour, supported by measured totals.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Bloom's Level: K5 – Evaluate

Recommend the final design for the metro.

Discuss:

- The types, constants and classes you recommend and why
- How a fare revision will be carried out
- The limitations of your design
- Whether the design remains correct for the whole network

Your recommendation must be based on evidence from your own experiments.

Alternative Concept and Complexity Analysis

If the fare table were read from a configuration file when the gate starts, instead of being compiled into the program, what would the solution be? Analyse the extra start-up time to read z fare values and the space they occupy, against the cost of recompiling and redeploying after every revision.

Evidence Expected

A technically justified recommendation supported by your tests and measurements.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate

Changed Requirement

The metro adds a fourth zone (Airport, Rs 60), a 10% weekend discount that does not combine with the peak surcharge, and a monthly pass that allows unlimited journeys in two chosen zones.

Re-evaluate your original design. You may RETAIN → MODIFY → or REDESIGN your original solution.

A. Impact Analysis – 2 Marks

Identify which constants, enum values, expressions, switch statements and classes are affected.

Evidence Expected

A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether the switch-based fare logic and the SmartCard class can absorb the changes, using worked examples.

Evidence Expected

A technically supported evaluation of the original design.

- Adaptation or Redesign – 2 Marks

  Describe and implement the revised design (for example: a fare table indexed by zone, a separate discount rule, a Pass class).

  Evidence Expected

  An appropriate design response with working code.

- Evidence-Based Defence – 2 Marks

  Defend your decision using test results and the trade-offs involved.

  Evidence Expected

  A well-supported final decision integrating tests and the changed requirements.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Object-Oriented Analysis	K5 – Evaluate	4
Task 2 – Tokens and Constants	K5 – Evaluate	3
Task 3 – Data Types and Compatibility	K5 – Evaluate	4

Task 4 – Implicit Conversions K5 – Evaluate 4

Task 5 – Operators and Precedence K5 – Evaluate 4

Task 6 – Control Structures K5 – Evaluate 4

Task 7 – Procedural vs Object-Oriented K5 – Evaluate 4

Task 8 – Input and Output K5 – Evaluate 4

Task 9 – Implementation & Testing K5 – Evaluate 4

Task 10 – Expectation vs Execution K5 – Evaluate 4

Task 11 – Final Recommendation K5 – Evaluate 4

Subtotal 42

Task 12 – Day-4 Design Challenge K5 – Evaluate 8

TOTAL 50

Submission Requirements

- GitHub repository link (mandatory) containing the executed code for every task, including every completed "Complete the Code" program, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including UML diagrams, pseudocode, test tables and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.

TASK 1 – Understand the Problem and Identify the Objects

1. Object-Oriented Analysis of the Metro Smart-Card Fare System

The City Metro Smart-Card System is used to calculate the fare for each passenger journey and deduct the fare from the passenger's smart card. From an object-oriented programming (OOP) point of view, the system can be divided into different objects. Each object represents a real-world entity and contains its own data and behaviour.

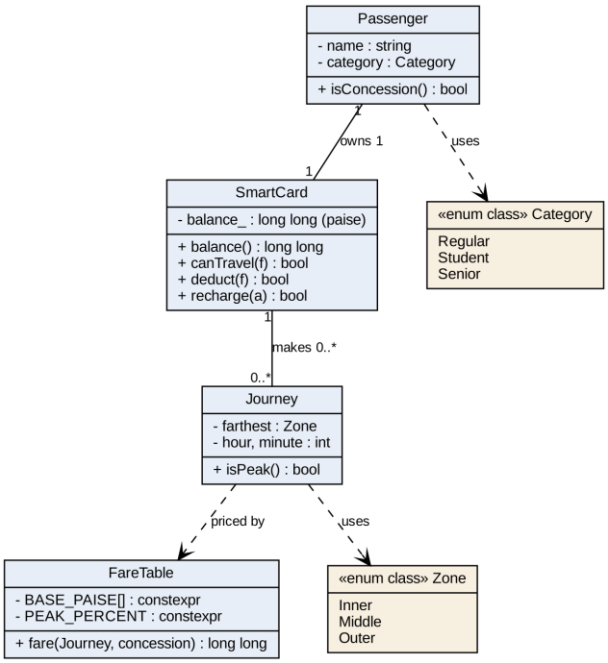
A. Objects, Their Data and Behaviour

Object	Data (Attributes)	Behaviour (Methods)	Justification
SmartCard	Card ID, balance, card type	Check balance, recharge card, deduct fare	A smart card stores the passenger's travel balance. It is responsible for maintaining the balance and ensuring that the fare is deducted correctly.
Passenger	Passenger ID, name, passenger category	Register passenger, identify passenger category	A passenger uses the smart card to travel. Passenger details help the system identify the type of passenger and apply relevant fare rules.
Journey	Source zone, destination zone, journey distance or zone combination, calculated fare	Calculate journey fare, validate journey, complete journey	A journey represents one metro trip. It stores the journey details required to calculate the correct fare.
Zone	Zone ID, zone name, zone details	Identify zone, provide zone information	Zones represent different geographical areas of the metro network. The source and destination zones help determine the applicable fare.
FareTable	Base fare, zone-wise fare values, applicable surcharge or discount values	Retrieve fare, calculate applicable charges, apply fare rules	The fare table stores the fare information separately from the smart card. This makes fare updates easier without changing the card's balance-management logic.

B. Comparison:

Procedural Programming	Object-Oriented Programming

Uses global variables and free functions.	Uses classes and objects.
Data and functions are managed separately.	Data and related functions are grouped together.
Data may be accessed by many functions.	Encapsulation controls access to object data.
Changes to a fare rule may affect multiple functions.	Fare-related changes can often be made mainly within the FareTable class.
Less suitable for managing many related entities.	Easier to organize and maintain a system with multiple entities.



C. Basic Principles of OOP in the Metro System

1. Encapsulation

Encapsulation means combining data and related functions inside a class and restricting direct access to important data.

In this system, the SmartCard class can keep the balance as a private variable. The balance can be accessed or modified through methods such as getBalance() and deductFare().

This prevents unauthorized changes to the card balance and helps ensure that fare deduction follows the system rules.

2. Abstraction

Abstraction means showing only the necessary operations to the user while hiding internal implementation details.

For example, a passenger only needs to tap the smart card to complete a journey. The passenger does not need to know how the system calculates the fare internally.

The Journey class can provide a method such as calculateFare() while hiding the detailed calculation process.

3. Inheritance

Inheritance allows one class to acquire the properties and behaviours of another class.

For example, a general Passenger class could be used as a base class, with derived classes such as RegularPassenger and StudentPassenger.

These classes can share common passenger details while allowing category-specific information or fare rules.

Assumption: These passenger categories are possible design extensions; the assignment does not specify exact concession rates.

4. Polymorphism

Polymorphism means that the same function name or interface can perform different operations depending on the object.

For example, a method named calculateFare() could be implemented differently for different passenger categories or fare policies.

This allows the system to support different fare calculations without changing the overall journey-processing structure.

D. Rules Every Correct Journey Must Satisfy

The following rules should be checked before completing a journey:

1. Minimum balance rule: After fare deduction, the smart-card balance must never fall below Rs. 50.
2. Valid card rule: The passenger must use a valid smart card.
3. Valid zone rule: The source and destination zones must be valid zones in the metro network.
4. Correct fare rule: The fare must be calculated according to the fare table and applicable fare conditions.
5. Sufficient balance rule: The card must have enough balance to pay the fare while maintaining the minimum balance of Rs. 50.
6. Correct deduction rule: The system must deduct the calculated fare only once for a completed journey.

2. Alternative Concept and Complexity Analysis

If the metro fare system were implemented using a purely procedural approach, it would use global variables and free functions instead of classes such as SmartCard, Journey and FareTable.

For example, the system could use global variables to store card balance, passenger type, source zone and destination zone. Separate functions would perform the operations:

- checkBalance() – checks the available card balance.
- calculateFare() – calculates the journey fare.
- validateJourney() – checks whether the journey is valid.
- deductFare() – deducts the fare from the card balance.

A. Time and Memory Complexity

Complexity	Procedural Approach	Object-Oriented Approach
Time per journey	O(1)	O(1)
Memory per card	O(1)	O(1)

B. Effect of Changing a Fare Rule

In procedural programming, fare-related logic may be written in multiple functions. If a fare rule changes, the programmer may need to modify several functions and check whether the change affects other parts of the program.

In object-oriented programming, fare-related calculations can be managed mainly inside the FareTable class. Therefore, a fare-rule change can often be made in one place without modifying the SmartCard class or other unrelated classes.

TASK 2 – Use Tokens, Keywords, Identifiers and Constants Correctly

1. Defining Fares, Surcharge and Limits

In the Metro Smart-Card System, fares, surcharges and balance limits should be declared in one place using properly named constants. This avoids repeating values throughout the program and makes future modifications easier.

The constexpr keyword is preferred for fixed fare values because it allows values to be evaluated at compile time and provides type checking.

A. Five Types of Tokens in C++

Token Type	Definition	Examples from the Metro System

Keywords	Reserved words that have predefined meanings in C++.	class, int, const, if, return
Identifiers	User-defined names given to variables, functions and classes.	SmartCard, baseFare, calculateFare
Constants	Fixed values that do not change during program execution.	50, 30.0, 100
Operators	Symbols used to perform operations.	+, -, *, /, >=, ==
Punctuators	Symbols used to structure and separate program statements.	{ }, ( ), ;, ,

B. Difference Between #define, const and constexpr

C++ provides different ways to define values that should not change.

1. #define

#define is a preprocessor directive used to define symbolic macros. The preprocessor replaces the macro name with its value before compilation.

Example:

#define INNER_FARE 10.0

#define MIDDLE_FARE 20.0

#define OUTER_FARE 30.0

#define MINIMUM_BALANCE 50.0

Advantages:

- Simple to use.
- Does not require a variable declaration.

Disadvantages:

- Does not provide normal C++ type checking.
- Does not follow normal variable scope rules.
- Can cause unexpected results because it performs text substitution.

2. const

The const keyword declares a variable whose value cannot be modified after initialization.

Example:

const double INNER_FARE = 10.0;

const double MIDDLE_FARE = 20.0;

const double OUTER_FARE = 30.0;

const double MINIMUM_BALANCE = 50.0;

Advantages:

- Provides type checking.
- Follows C++ scope rules.
- Prevents accidental modification.

3. constexpr

The constexpr keyword declares a value that can be evaluated at compile time when its initialization satisfies constant-expression requirements.

Example:

constexpr double INNER_FARE = 10.0;

constexpr double MIDDLE_FARE = 20.0;

constexpr double OUTER_FARE = 30.0;

constexpr double PEAK_SURCHARGE_PERCENT = 20.0;

constexpr double MINIMUM_BALANCE = 50.0;

Advantages:

- Provides type safety.
- Supports compile-time evaluation.
- Makes fixed values easier to maintain.
- Follows normal C++ scope rules.

Why constexpr is preferred for the fare table:

The base fares, peak surcharge percentage and minimum balance are fixed values known before the program runs. Therefore, constexpr is suitable because these values can be evaluated at compile time. If a fare changes, the programmer only needs to update its declaration in one place.

C. Using enum and enum class for Metro Zones

An enumeration is a user-defined type that represents a fixed set of named values.

1. enum

A traditional enum can define the available metro zones.

```
enum Zone {
  INNER,
   MIDDLE,
   OUTER
};
```

2. enum class

An enum class provides stronger type safety and keeps its enumerator names within the scope of the enumeration.

```
enum class Zone {
INNER,
MIDDLE,
   OUTER


};
```

Usage:

```
Zone source = Zone::INNER;
Zone destination = Zone::MIDDLE;
```

Why enum class is safer:

- It prevents accidental implicit conversion to integers.
- It requires qualified names such as Zone::INNER.
- It reduces naming conflicts.
- It improves code readability and type safety.

Therefore, enum class is preferred for representing metro zones.

D. Naming Rules and Conventions for Identifiers

1. An identifier can contain letters, digits and underscores.
2. It must not begin with a digit.
3. Spaces and special characters are not allowed.
4. C++ keywords cannot be used as identifiers.

5. Identifiers are case-sensitive.
6. Meaningful names should be used to improve readability.

Examples:

Identifier	Valid or Invalid	Reason
baseFare	Valid	Meaningful variable name
MINIMUM_BALANCE	Valid	Valid constant name
calculateFare	Valid	Meaningful function name
2zone	Invalid	Begins with a digit
class	Invalid	C++ keyword
base fare	Invalid	Contains a space

1.Naming conventions used in this system:

- Class names: SmartCard, FareTable, Journey
- Variable names: cardBalance, sourceZone
- Function names: calculateFare(), deductFare()
- Constants: BASE_FARE, MINIMUM_BALANCE
- Enumeration values: Zone::INNER, Zone::MIDDLE, Zone::OUTER

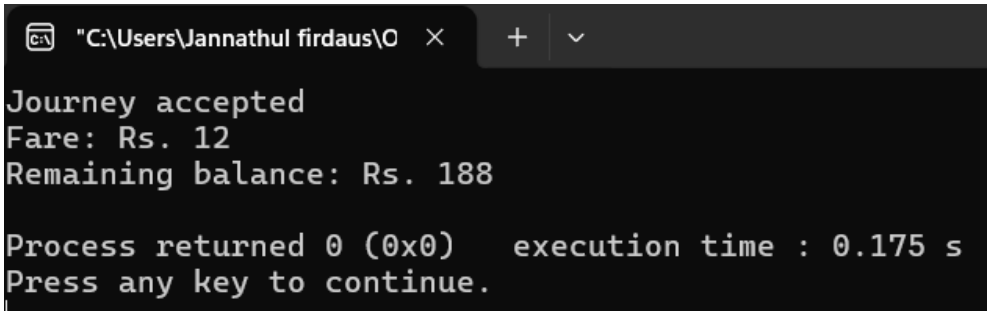
2. Example: Centralized Fare Constants and Zone Definition

The following code demonstrates how fare values and zone names can be defined in one place.

```
#include <iostream>
using namespace std;
// Fixed fare-related constants
constexpr double INNER_FARE = 10.0;
constexpr double MIDDLE_FARE = 20.0;
constexpr double OUTER_FARE = 30.0;
constexpr double PEAK_SURCHARGE_PERCENT = 20.0;
constexpr double MINIMUM_BALANCE = 50.0;
// Metro zone enumeration
enum class Zone {
```

```
    INNER,
    MIDDLE,
    OUTER
};
int main() {
    double balance = 200.0;
    double fare = INNER_FARE;
    // Apply 20% peak surcharge
    bool peakHour = true;
    if (peakHour) {
        fare += fare * PEAK_SURCHARGE_PERCENT / 100.0;
    }

    if (balance - fare >= MINIMUM_BALANCE) {
        balance -= fare;
        cout << "Journey accepted" << endl;
        cout << "Fare: Rs. " << fare << endl;
        cout << "Remaining balance: Rs. "
             << balance << endl;
    }
    else {
        cout << "Insufficient balance" << endl;
    }
    return 0;
}
```



3. Alternative Concept and Complexity Analysis

If fares were defined using #define macros instead of constexpr constants and enum class, the program would behave differently in terms of type checking, scope and safety.

A. Comparison of #define and constexpr

Feature	**#define**	**constexpr**
Processing	Replaced by the preprocessor	Handled by the C++ compiler
Type checking	No normal type checking	Type checking is provided
Scope	Not governed by normal C++ scope rules	Follows C++ scope rules
Compile-time evaluation	Text substitution	Compile-time evaluation when valid
Memory usage	Macro itself occupies no storage	Usually requires no storage if used as a compile-time value
Safety	Less type-safe	More type-safe

B. Comparison of enum and enum class

Feature	**enum**	**enum class**
Type safety	Less strict	Strong type safety
Integer conversion	Can implicitly convert to integer	No implicit conversion to integer
Scope	Enumerator names can enter the surrounding scope	Enumerator names are scoped
Naming conflicts	More likely	Less likely
Recommended use	Simple enumerations	Safer zone representation

C. Memory and Running-Time Analysis

Memory usage:

- A #define macro is replaced before compilation and does not occupy memory as a variable.
- A constexpr value usually does not require separate storage when used as a compile-time constant, although storage may be required if its address is taken or an object must be materialized.
- An enum class defines a type; an actual variable of that type occupies memory according to its underlying type.

Running time:

Using #define or constexpr for fixed fare values generally does not change the running time of the fare calculation. Both can result in constant values being used directly in the compiled program.

Therefore, the choice of constexpr is mainly for type safety, scope and maintainability rather than improving the time complexity.

TASK 3 – Choose Data Types and Check Type Compatibility

1. Selection of C++ Data Types

In the Metro Smart-Card System, appropriate data types must be selected for fares, card balances, passenger details, journey times, zone numbers and journey counts. Correct data-type selection helps prevent overflow, loss of precision and unnecessary memory usage.

A. Data-Type Selection Table

System value	**Recommended C++ type**	**Reason**
Fare	long long	Stores fare in integer paise without decimal precision errors.
Card balance	long long	Stores the balance in paise and supports large values.
Passenger name initial	char	Stores a single character.
Passenger ID	int	Suitable for ordinary numeric identification values.
Zone number	int or enum class Zone	Represents a zone number or a fixed set of named zones.
Journey time	int	Can store time as minutes from midnight or seconds from a reference point.
Journey count	unsigned int	Suitable for a count that cannot be negative.

Daily collection	long long	Can store a large total collected across the network.
Discount percentage	double	Can represent percentage calculations when required.
Journey validity	bool	Stores either true or false.
Fixed fare constants	constexpr long long	Stores compile-time fare values in paise.
Automatically inferred local value	auto	Allows the compiler to infer the type from the initialization expression.

Justification: Integer types are preferred for money because fare and balance values must be exact. bool is suitable for validation results, while double is useful for percentage calculations. unsigned int can be used for journey counts because negative counts are not meaningful.

2. Why Money Should Be Stored in Integer Paise Instead of Double Rupees

Money should be stored as integer paise rather than double rupees because floating-point values cannot represent every decimal fraction exactly.

For example:

- Rs. 10.50 is stored as 1050 paise.
- Rs. 25.75 is stored as 2575 paise.
- Rs. 100.00 is stored as 10000 paise.

Using integer paise avoids floating-point representation errors during fare addition and subtraction.

Example of the problem with double:

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
   double a = 0.1;
   double b = 0.2;
   cout << setprecision(17) << (a + b) << endl;
   return 0;
}
```

The output may be:

0.30000000000000004

Correct solution using integer paise:

```
#include <iostream>

using namespace std;

int main() {

   long long a = 10;  // 10 paise

   long long b = 20;  // 20 paise

   long long total = a + b;

   cout << "Total = " << total << " paise";

   return 0;

}
```

Output:

Total = 30 paise

3. Type Compatibility in C++

Type compatibility determines whether different data types can be used together in an expression.

C++ performs implicit type conversion when different data types are used in the same expression. This is also known as type promotion or usual arithmetic conversion.

A. Mixing int, double and bool

Example:

```
#include <iostream>

using namespace std;

int main() {

   int fare = 20;

   double surcharge = 5.50;

   bool valid = true;

   double total = fare + surcharge + valid;

   cout << total;

   return 0;

}
```

Output:

26.5

4. Sizes and Ranges of Data Types

The sizeof operator is used to find the memory occupied by a data type in bytes. The exact sizes depend on the compiler and computer architecture.

Data type	Typical size	Range or possible values	Use in metro system
bool	1 byte	true or false	Journey validity
char	1 byte	Usually -128 to 127, or 0 to 255	Passenger initial
int	4 bytes	-2,147,483,648 to 2,147,483,647	Zone number, passenger ID
unsigned int	4 bytes	0 to 4,294,967,295	Journey count
long long	8 bytes	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Fare, balance and daily collection
float	4 bytes	Approximately 6–7 decimal digits of precision	Approximate measurements, not money
double	8 bytes	Approximately 15–16 decimal digits of precision	Percentage calculations
auto	Depends on inferred type	Depends on inferred type	Automatically deduced variables

A. Program to Check Data-Type Sizes

The following program can be compiled and executed to find the actual sizes on the system used for the assignment.

```
#include <iostream>

using namespace std;

int main() {

    cout << "Size of bool: " << sizeof(bool) << " bytes\n";

    cout << "Size of char: " << sizeof(char) << " bytes\n";

    cout << "Size of int: " << sizeof(int) << " bytes\n";

    cout << "Size of unsigned int: "
```

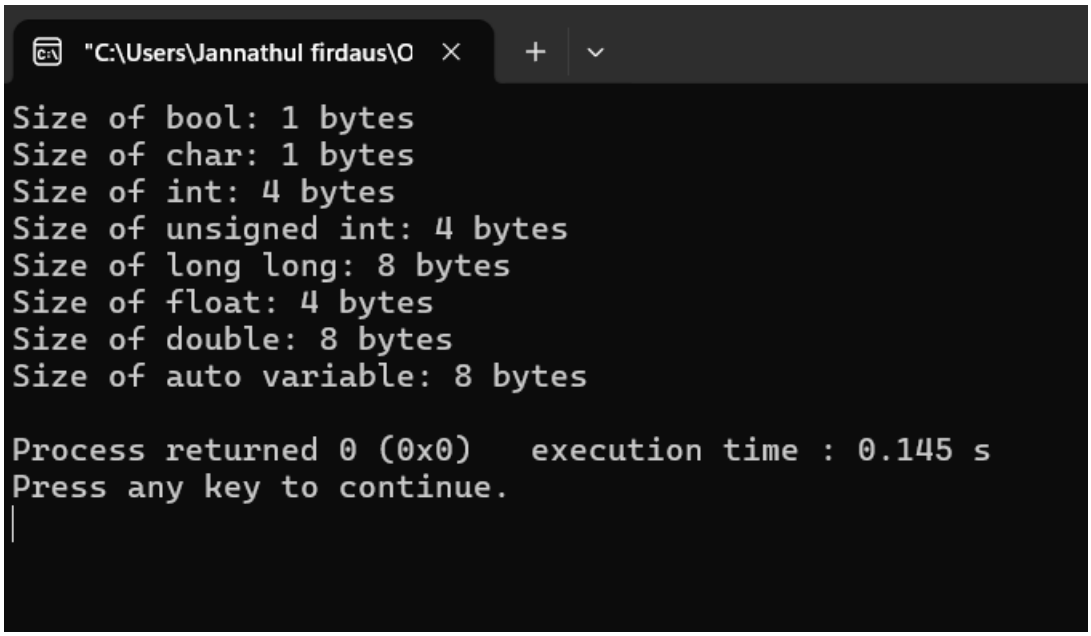
```
        << sizeof(unsigned int) << " bytes\n";
    cout << "Size of long long: "
        << sizeof(long long) << " bytes\n";
    cout << "Size of float: " << sizeof(float) << " bytes\n";
    cout << "Size of double: " << sizeof(double) << " bytes\n";

    auto fare = 2000LL;
    cout << "Size of auto variable: "
        << sizeof(fare) << " bytes\n";

    return 0;
}
```

output on a typical 64-bit GCC system:

Program to Check Data-Type Sizes



5. Ranges from <climits> and Daily Collection

The C++ header file <climits> provides constants that describe the minimum and maximum values supported by integer types.

Important constants include:

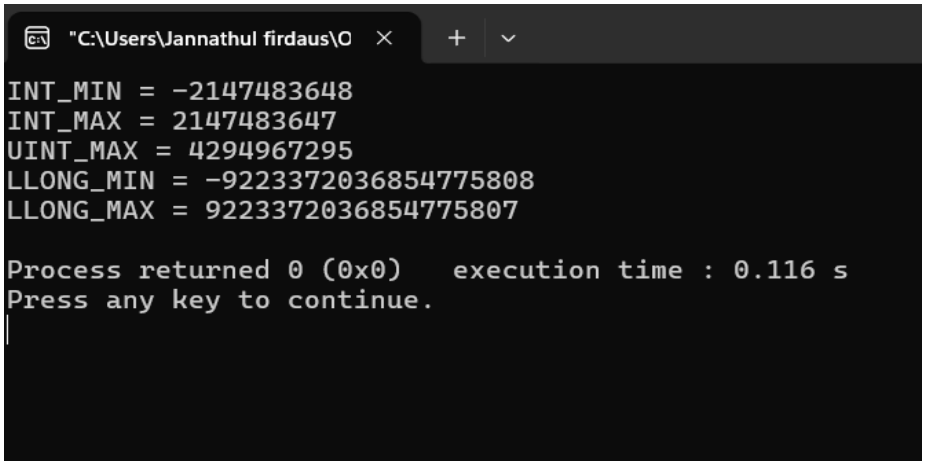
Constant	Meaning

INT_MIN	Minimum value of int
INT_MAX	Maximum value of int
UINT_MAX	Maximum value of unsigned int
LLONG_MIN	Minimum value of long long
LLONG_MAX	Maximum value of long long

A. Program to Check Integer Ranges

```
#include <iostream>
#include <climits>
using namespace std;
int main() {
    cout << "INT_MIN = " << INT_MIN << endl;
    cout << "INT_MAX = " << INT_MAX << endl;
    cout << "UINT_MAX = " << UINT_MAX << endl;
    cout << "LLONG_MIN = " << LLONG_MIN << endl;
    cout << "LLONG_MAX = " << LLONG_MAX << endl;
    return 0;
}
```

output:



B. Is int Enough for One Day's Metro Collection?

For a typical 32-bit int, the maximum positive value is 2,147,483,647.

If money is stored in paise, the maximum amount that a signed int can represent is:

2,147,483,647 /100 = Rs.21,474,836.47

This means an int can store a maximum of approximately Rs. 2.14 crore in paise.

However, the total daily collection depends on the number of journeys and the average fare. Since the assignment does not specify the daily passenger count, it is not possible to guarantee that int will always be sufficient.

The daily collection can be calculated as:

Daily Collection = Number of Journeys X Average Fare in Paise

Therefore, long long is a safer choice for storing the total daily collection because it supports a much larger range than int.

6. Alternative Concept: Storing Money as double Rupees

If money were stored as double rupees instead of long long paise, the fare and balance would contain decimal values.

For example:

double fare = 20.50;

double balance = 200.00;

balance = balance - fare;

This approach is convenient because money can be written directly in rupees and paise. However, floating-point representation can introduce small rounding errors.

A. Time and Memory Complexity Comparison

Factor	**long long paise**	**double rupees**
Addition time	O(1)	O(1)
Space per value	Typically 8 bytes	Typically 8 bytes
Precision	Exact for integer paise within range	May have rounding errors
Suitability for money	Recommended	Less suitable for exact financial calculations

B. How Rounding Error Grows When Many Fares Are Added

Program to demonstrate the rounding error:

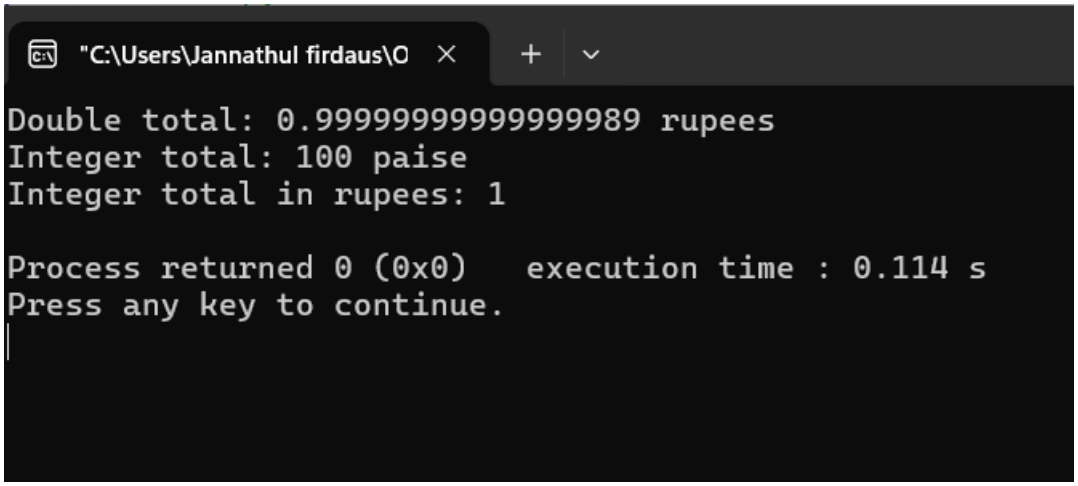
#include <iostream>

#include <iomanip>

using namespace std;

```
int main() {
    double totalRupees = 0.0;
    long long totalPaise = 0;
    for (int i = 0; i < 10; i++) {
        totalRupees += 0.1;
        totalPaise += 10;
    }
    cout << setprecision(17);
    cout << "Double total: "
         << totalRupees << " rupees" << endl;
    cout << "Integer total: "
         << totalPaise << " paise" << endl;
    cout << "Integer total in rupees: "
         << totalPaise / 100.0 << endl;
    return 0;
}
```

result:



C. Resolution of the Problem

To prevent rounding errors:

1. Store all fares and balances as integer paise using long long.
2. Convert rupees to paise when accepting monetary input.

3. Perform fare addition and subtraction using integer arithmetic.
4. Convert paise to rupees only when displaying the result.

TASK 4 – Control Implicit Conversions

1. Introduction

Implicit conversion in C++ occurs when the compiler automatically converts a value from one data type to another without an explicit instruction from the programmer.

In the Metro Smart-Card System, implicit conversions may occur during fare calculations, balance comparisons, percentage calculations and input validation.

Understanding these conversions is important because unexpected conversions can cause incorrect fare calculations, loss of precision and logical errors.

2. Different Types of Implicit Conversions

A. Integer Division and Integer-to-Double Conversion

When two integer values are divided, C++ performs integer division. The fractional part is discarded.

Example:

int a = 7;

int b = 2;

double c = a / b;

Here, a / b is evaluated first using integer division.

7 / 2 = 3

The result 3 is then implicitly converted into double, producing 3.0.

Therefore, c stores 3.0, not 3.5.

Correct method:

double c = static_cast<double>(a) / b;

Now, a is explicitly converted into double before division.

7.0 / 2 = 3.5

This produces the correct decimal result.

B. Comparison of Signed and Unsigned Values

C++ supports signed integers, which can store positive and negative values, and unsigned integers, which can store only non-negative values.

When a signed integer is compared with an unsigned integer of the same rank, the signed value may be converted to unsigned.

Example:

```
unsigned int u = 1;

int s = -1;

if (s < u) {

   cout << "s < u";

}

else {

   cout << "s >= u";

}
```

Although s contains -1, it is converted to an unsigned value for comparison. This produces a very large positive value, so the condition is false.

The output is:

```
s >= u
```

C. Narrowing Conversions

Narrowing conversion occurs when a value is converted into a type that cannot represent the original value without losing information.

For example:

```
double fare = 25.75;

int amount = fare;
```

The fractional part is discarded, so amount becomes 25.

C++ brace initialization helps prevent certain narrowing conversions.

```
int x {3.7};  // Compilation error
```

The compiler rejects this initialization because converting 3.7 from double to int would lose the fractional part.

Correct initialization:

```
int x{3};
```

This is accepted because the value can be represented exactly as an integer.

D. static_cast and Explicit Conversion

static_cast is used when the programmer wants to explicitly convert a value from one type to another.

It makes the conversion visible and improves code readability.

Example:

```
int farePaise = 2050;

double fareRupees =

   static_cast<double>(farePaise) / 100;

cout << fareRupees;
```

Output:

20.5

Here, farePaise is explicitly converted from int to double before division. This ensures that the division produces a decimal result.

static_cast is useful when:

- Integer division must produce a decimal result.
- A numeric value must be converted into another numeric type.
- A conversion should be clearly documented in the program.

However, static_cast does not automatically make an unsafe conversion safe. The programmer must still ensure that the destination type can represent the value correctly.

3. Predict the Output Before Running the Program

A. My Predicted Output (Before Running)

3 3.5

C 68

1 true

s >= u

4. Corrected Code Using Explicit Conversions and Brace Initialization

The following version makes conversions more visible and avoids the unexpected signed/unsigned comparison.

```
#include <iostream>

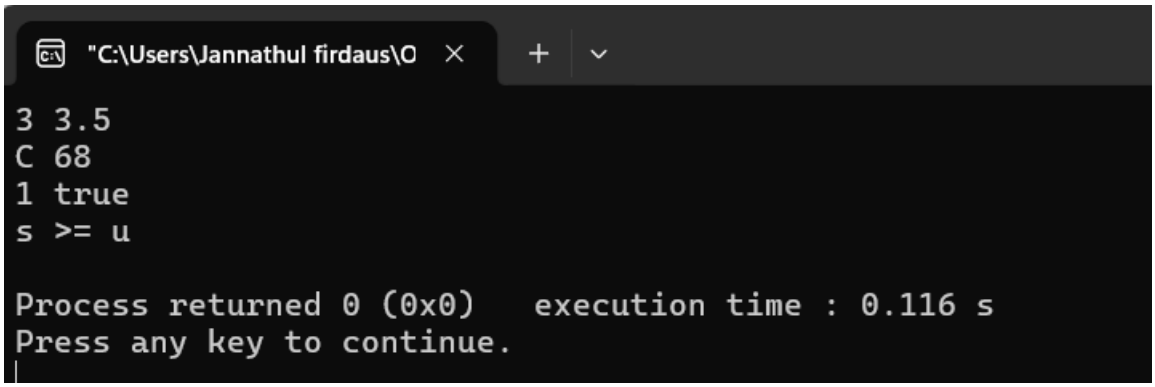
using namespace std;

int main() {
```

```
int a{7};
int b{2};
// Explicit conversion after integer division
double c{static_cast<double>(a / b)};
// Explicit conversion before division
double d{
   static_cast<double>(a) /
   static_cast<double>(b)
};
cout << c << ' ' << d << '\n';
// Explicit conversion to char
char ch{static_cast<char>('A' + 2)};
cout << ch << ' '
    << static_cast<int>(ch) + 1 << '\n';
// Explicit conversion from int to bool
bool flag{static_cast<bool>(5)};
cout << static_cast<int>(flag) << ' '
    << boolalpha << flag << '\n';
unsigned int u{1};
int s{-1};
// Check negative values before unsigned conversion
if (s < 0) {
   cout << "s >= u" << '\n';
}
else if (static_cast<unsigned int>(s) < u) {
   cout << "s < u" << '\n';
}
else {
   cout << "s >= u" << '\n';
```

```
    }
    return 0;
}
```

Output:



5. Alternative Concept and Complexity Analysis

If every conversion in the program were written explicitly using static_cast and brace initialization, the programmer would have greater control over type conversions.

A. Comparison

Implicit conversion	**Explicit conversion**
Compiler automatically converts values.	Programmer specifies the conversion using static_cast.
Can hide unexpected data loss.	Makes conversions visible.
May produce signed/unsigned comparison surprises.	Encourages deliberate handling of signed and unsigned values.
Some narrowing errors may go unnoticed.	Brace initialization rejects many narrowing conversions at compile time.
Less code may be required.	More explicit code, but easier to review.

B. Running Time and Memory

Factor	**Implicit conversions**	**Explicit conversions**
Running time	O(1)	O(1)
Memory usage	O(1)	O(1)

Type checking	Compiler performs automatic conversions	Programmer specifies conversions and compiler checks them
Error detection	Some problems may appear only through incorrect results	Many narrowing mistakes can be detected during compilation

C. Errors Caught at Compile Time

Brace initialization helps the compiler identify narrowing conversions.

Example:

int x {3.7};

This produces a compilation error because a decimal value is being narrowed to an integer.

Similarly, using static_cast makes a conversion intentional, but it does not guarantee that the result is safe.

This is valid C++, but the fractional part is lost and x becomes 3.

Therefore, explicit conversion and brace initialization improve code safety, but the programmer must still check whether the conversion is logically correct.

TASK 5 – Write Correct Expressions with the Right Precedence

1. Expressions for Peak Check, Surcharge and Concession

In the Metro Smart-Card System, operators are used to check peak hours, calculate surcharges and apply passenger concessions. Correct operator precedence and associativity are important to ensure that fare calculations produce the expected results.

A. Peak-Hour Check

Peak hours are assumed to be from 8:00 AM to 10:00 AM and 5:00 PM to 8:00 PM.

The correct C++ expression is:

(hour >= 8 && hour < 10) ||

(hour >= 17 && hour < 20)

Explanation:

- >= and < compare the hour with the peak-hour limits.
- && requires both conditions in each time range to be true.
- || checks whether either of the two peak periods is active.
- Parentheses make the two time ranges clear and easy to understand.

B. Surcharge Calculation

A surcharge can be added to the base fare during peak hours.

Example:

```
double fare = baseFare;
if (isPeakHour) {
    fare += peakSurcharge;
}
```

Here, += is a compound assignment operator. It adds the surcharge to the existing fare and stores the result back in fare.

It is equivalent to:

```
fare = fare + peakSurcharge;
```

C. Concession Calculation

A concession can be applied to the base fare for an eligible passenger.

Example:

```
double finalFare =
    eligible ? baseFare * (1.0 - concessionRate)
             : baseFare;
```

Explanation:

- ? : is the conditional (ternary) operator.
- If eligible is true, the discounted fare is calculated.
- Otherwise, the original base fare is used.
- * has higher precedence than the conditional operator.
- Parentheses make the discount calculation clear.

2. Operator Precedence and Associativity

Operator	**Purpose**	**Associativity**
++	Increment	Prefix: right to left; Postfix: left to right
*, /, %	Multiplication, division, remainder	Left to right
+, -	Addition, subtraction	Left to right

<, >, <=, >=	Relational comparison	Left to right
==, !=	Equality comparison	Left to right
&&	Logical AND	Left to right
||	Logical OR	Left to right
?:	Conditional operator	Right to left
=, +=, -=, *=	Assignment	Right to left

Example:

int result = 5 + 3 * 2;

Therefore, the result is 11.

3. Difference Between x++ and ++x

Both x++ and ++x increase the value of x by one, but they differ in the value produced by the expression.

Post-increment (x++):

- Uses the current value first.
- Increments the variable afterwards.

Pre-increment (++x):

- Increments the variable first.
- Uses the updated value.

Example:

int x = 5;

cout << x++ << endl; // Prints 5

cout << ++x << endl; // Prints 7

After the first statement, x becomes 6. The second statement increments it to 7 before printing.

Why i = i++ Is a Mistake

int i = 5;

i = i++;

In modern C++, this expression has undefined behaviour because the modification and value computation of i are not safely sequenced relative to the assignment.

The correct way to increment is:

i++;

or:

++i;

These statements are clear and avoid unnecessary assignment.

4. Predict the Output Before Running the Program

A. My Predicted Output (Before Running)

After evaluating the expressions according to C++ operator precedence and associativity, my predicted output is:

11 16

9 6 4

6 4

3 1 -3 -1

18

B. Predicted Output vs Actual Output

Output line	Predicted output	Expected actual output
1	11 16	11 16
2	9 6 4	9 6 4
3	6 4	6 4
4	3 1 -3 -1	3 1 -3 -1
5	18	18

5. Alternative Concept and Complexity Analysis: Lookup Table for Peak Hours

Instead of checking peak hours using multiple && and || operators, a Boolean lookup table can be used.

A lookup table stores whether each hour of the day is a peak hour.

A. Implementation Using a Boolean Lookup Table

#include <iostream>

```
using namespace std;
int main() {
    bool peak[24] = {};
    // Morning peak: 8 AM to 9:59 AM
    for (int i = 8; i < 10; i++) {
        peak[i] = true;
    }
    // Evening peak: 5 PM to 7:59 PM
    for (int i = 17; i < 20; i++) {
        peak[i] = true;
    }
    int hour = 9;
    if (hour >= 0 && hour < 24 && peak[hour]) {
        cout << "Peak hour";
    }
    else {
        cout << "Non-peak hour";
    }
    return 0;
}
```

Output:

Peak hour

The range check is important because accessing an array with an invalid hour can cause undefined behaviour.

TASK 6 – Design the Control Structures

1. Choosing the control structures

- if–else: Used to check whether the passenger has sufficient balance before deducting the fare.
- switch: Used to select the base fare according to the travel zone (Inner, Middle, or Outer).

- Conditional operator (?:): Can be used for a simple decision, such as selecting a fare based on whether peak hours apply.
- for loop: Best suited for processing a fixed list of journeys.
- Range-based for loop: Suitable for processing each journey in an array without using an index. It is used in this program.
- while loop: Suitable when journeys are processed until a condition becomes false.
- break: Used to exit a loop or switch when required.
- continue: Used to skip an invalid journey and process the next one.
- No matching switch case: The default case handles an unmatched zone. If no default is provided, no case body executes.

2. Pseudocode for fare computation

Algorithm: CalculateMetroFare

Input: Zone, peak-hour status, card balance

Output: Fare and updated card balance

Begin

Read zone and peak-hour status

Switch zone

Case Inner:

baseFare = 1000

Case Middle:

baseFare = 2000

Case Outer:

baseFare = 3000

Default:

Return invalid fare

End Switch

If peak-hour status is true then

fare = baseFare + (baseFare * 20 / 100)

Else

fare = baseFare

```
    End If
    If balance - fare >= 5000 then
        balance = balance - fare
        Display fare and balance
    Else
        Display "Denied (low balance)"
    End If
End
```

3. Completed C++ program

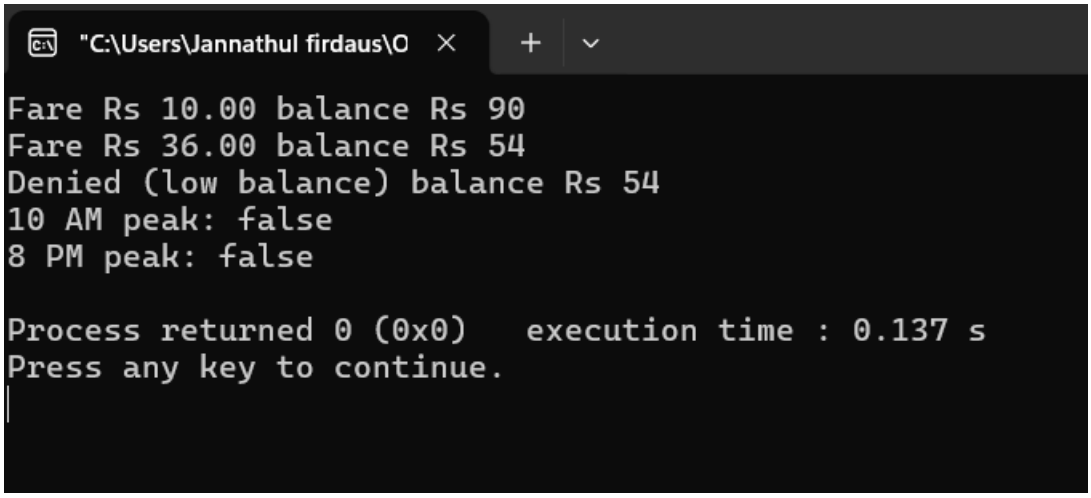
```
#include <iomanip>
#include <iostream>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak)
{
    long long baseFare = 0;
    // TODO 1
    switch (z) {
        case Zone::Inner:
            baseFare = BASE_PAISE[0];
            break;
        case Zone::Middle:
            baseFare = BASE_PAISE[1];
            break;
        case Zone::Outer:
            baseFare = BASE_PAISE[2];
```

```
            break;
        default:
            return -1;
    }
    if (peak) {
        baseFare += baseFare * PEAK_SURCHARGE_PERCENT / 100;
    }
    return baseFare;
}
bool canTravel(long long balance, long long f)
{
    // TODO 2
    return f >= 0 && balance - f >= MIN_BALANCE_PAISE;
}
bool isPeak(int hour)
{
    return (hour >= 8 && hour < 10) ||
           (hour >= 17 && hour < 20);
}
int main()
{
    long long balance = 10000;

    struct Trip {
        Zone zone;
        bool peak;
    };
    Trip trips[] = {
        {Zone::Inner, false},
```

```
        {Zone::Outer, true},
        {Zone::Middle, false}
    };
    for (const Trip& t : trips) {
        long long f = fare(t.zone, t.peak);
        if (canTravel(balance, f)) {
            balance -= f;
            cout << "Fare Rs " << f / 100 << '.'
                 << setw(2) << setfill('0') << f % 100
                 << setfill(' ');
        } else {
            cout << "Denied (low balance)";
        }
        cout << " balance Rs " << balance / 100 << '\n';
    }
    cout << "10 AM peak: " << boolalpha << isPeak(10) << '\n';
    cout << "8 PM peak: " << boolalpha << isPeak(20) << '\n';

    return 0;
}
```



5. Alternative concept and complexity analysis

Using an array instead of a switch:

```
constexpr long long FARE[] = {1000, 2000, 3000};

long long baseFare = FARE[static_cast<int>(z)];
```

Factor	Switch statement	Array lookup
Time per lookup	O(1)	O(1)
Space	O(1) additional space	O(1), three fare values
Adding a fourth zone	Add another case	Add another array value

Both methods provide constant-time fare lookup. A switch may be implemented as a jump table by the compiler. An array provides direct indexing but requires valid zone indices.

TASK 7 – From Procedural to Object-Oriented (4 Marks)

1. Procedural version

```
#include <iostream>

using namespace std;

const long long MIN_BALANCE = 5000;

bool canTravel(long long balance, long long fare)

{

   return balance - fare >= 0 && balance - fare >= MIN_BALANCE;

}

void deduct(long long &balance, long long fare)

{

   if (canTravel(balance, fare))

      balance -= fare;

}

int main()

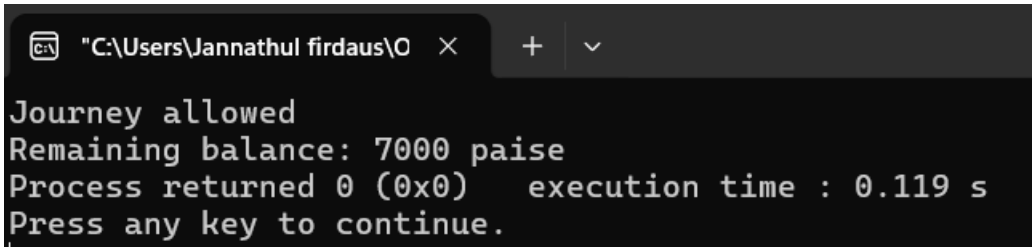
{

   long long balance = 10000;

   long long fare = 3000;


   if (canTravel(balance, fare)) {
```

```
        deduct(balance, fare);
        cout << "Journey allowed\n";
    }
    else {
        cout << "Journey denied\n";
    }
    cout << "Remaining balance: " << balance << " paise";
    return 0;
}
```



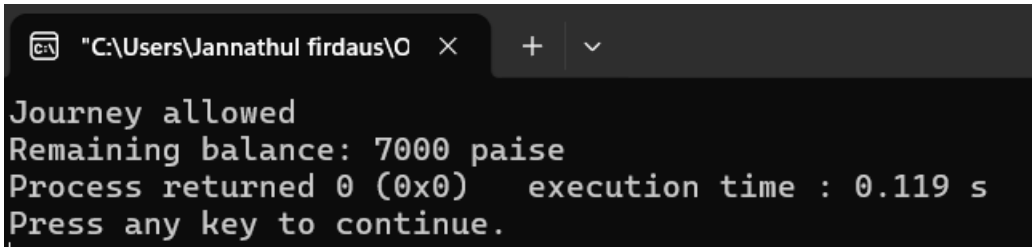
2. Object-oriented version

In the object-oriented approach, the SmartCard class keeps the balance private and provides public functions to check and deduct the fare.

```
#include <iostream>
using namespace std;
class SmartCard {
private:
    long long balance;
    static constexpr long long MIN_BALANCE = 5000;
public:
    SmartCard(long long initialBalance) {
        balance = (initialBalance >= MIN_BALANCE ? initialBalance : MIN_BALANCE;
    }
    bool canTravel(long long fare) const {
        return fare >= 0 && balance - fare >= MIN_BALANCE;
    }
```

```
    bool deduct(long long fare) {
        if (canTravel(fare)) {
            balance -= fare;
            return true;
        }
        return false;
    }
    long long getBalance() const {
        return balance;
    }
};
int main()
{
    SmartCard card(10000);
    long long fare = 3000;
    if (card.deduct(fare))
        cout << "Journey allowed\n";
    else
        cout << "Journey denied\n";

    cout << "Remaining balance: "
         << card.getBalance() << " paise";
    return 0;
}
```



3. Comparison of procedural and object-oriented approaches

Factor	Procedural approach	Object-oriented approach
Data storage	Separate balance variable	Private balance inside class
Functions	Free functions	Class member functions
Data protection	Less protection	Encapsulation protects balance
Negative balance	Must be checked by functions	Checked through class methods
Minimum balance change	Update the global constant	Update the class constant
Maintenance	Less organized for larger systems	Easier to maintain and extend

4. How the class prevents a negative balance

The balance variable is declared as private, so it cannot be modified directly from outside the class. The deduct() function checks canTravel() before deducting the fare. It verifies that the fare is non-negative and that the remaining balance will be at least ₹50. Therefore, an invalid deduction is rejected, and the minimum balance requirement is maintained.

5. Alternative concept and complexity analysis

If the balance were declared as a public data member:

public:

long long balance;

Any part of the program could directly modify the balance without calling deduct() or canTravel().

Factor	Private balance	Public balance
Time complexity	O(1)	O(1)
Space complexity	O(1)	O(1)
Data protection	Protected by encapsulation	No protection
System rule	Minimum balance can be enforced through class methods	Any code can bypass the minimum balance rule

TASK 8 – Read and Print Data Safely (4 Marks)

1. Reading input safely using cin

The cin statement reads user input. If the user enters letters instead of numbers, cin.fail() detects the invalid input.

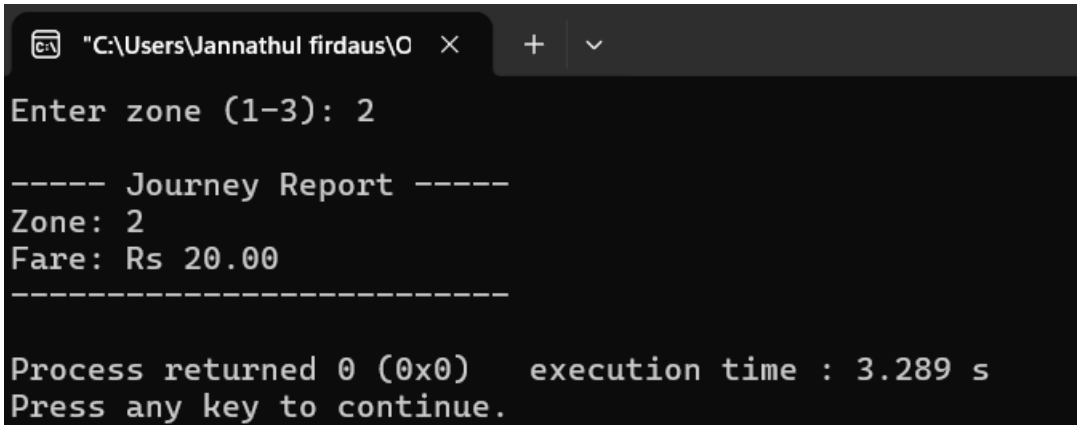
- cin.fail() checks whether input failed.
- cin.clear() clears the error state.
- cin.ignore() discards the invalid input from the input buffer.

2. C++ program for safe input and formatted output

```
#include <iostream>
#include <iomanip>
#include <limits>
using namespace std;
int main()
{
   int zone;
   long long fare = 1000;
   cout << "Enter zone (1-3): ";
   while (!(cin >> zone) || zone < 1 || zone > 3) {
      cout << "Invalid input. Enter zone (1-3): ";
      if (cin.fail()) {
         cin.clear();
         cin.ignore(numeric_limits<streamsize>::max(), '\n');
      }
   }
   if (zone == 1)
      fare = 1000;
   else if (zone == 2)
      fare = 2000;
   else
     fare = 3000;
```

```
    cout << "\n----- Journey Report -----\n";
    cout << "Zone: " << zone << '\n';
    cout << "Fare: Rs "
         << fixed << setprecision(2)
         << fare / 100.0 << '\n';

    cout << "--------------------------\n";
    return 0;
}
```



3. Formatting output

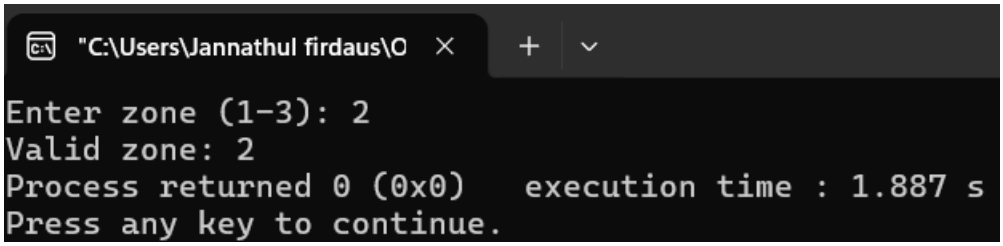
Manipulator	**Purpose**
setw()	Sets the width of the next output field
setfill()	Sets the character used to fill empty spaces
fixed	Displays floating-point values in fixed decimal format
setprecision(2)	Displays two digits after the decimal point

4. Alternative concept: getline() and stoi()

Instead of using cin >>, input can be read as a complete line using getline() and converted into an integer using stoi().

```
#include <iostream>
#include <string>
using namespace std;
int main()
```

```
{
    string input;
    int zone;
    cout << "Enter zone (1-3): ";
    getline(cin, input);
    try {
        size_t pos;
        zone = stoi(input, &pos);
        if (pos != input.length() || zone < 1 || zone > 3)
            throw invalid_argument("Invalid zone");
        cout << "Valid zone: " << zone;
    }
    catch (const exception&) {
        cout << "Invalid input";
    }
    return 0;
}
```



5. Complexity analysis and comparison

Factor	**cin >>**	**getline() with stoi()**
Time complexity	O(L)	O(L)
Buffer space	Input buffer	O(L) for the input string
Invalid input recovery	clear() and ignore()	try-catch handles conversion errors
Input reading	Reads formatted values	Reads the complete line

Here, L represents the length of the input line.

TASK 9 – Implement and Test the Fare Logic

1. Complete C++ program

The program uses the following fare rules:

- Inner Zone: ₹10, Middle Zone: ₹20, Outer Zone: ₹30.
- Peak hours: 8:00–10:00 and 17:00–20:00.
- Peak surcharge: 20%.
- Minimum balance after travel: ₹50.
- Negative recharge and invalid zone or hour values are rejected.

```
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
using namespace std;
const long long MIN_BALANCE = 5000;
const int BASE_FARE[] = {1000, 2000, 3000};
bool isPeak(int hour)
{
   return (hour >= 8 && hour < 10) ||
          (hour >= 17 && hour < 20);
}
long long calculateFare(int zone, int hour)
{
   if (zone < 1 || zone > 3 || hour < 0 || hour > 23)
      return -1;

   long long fare = BASE_FARE[zone - 1];
   if (isPeak(hour))
      fare += fare * 20 / 100;
   return fare;
}
```

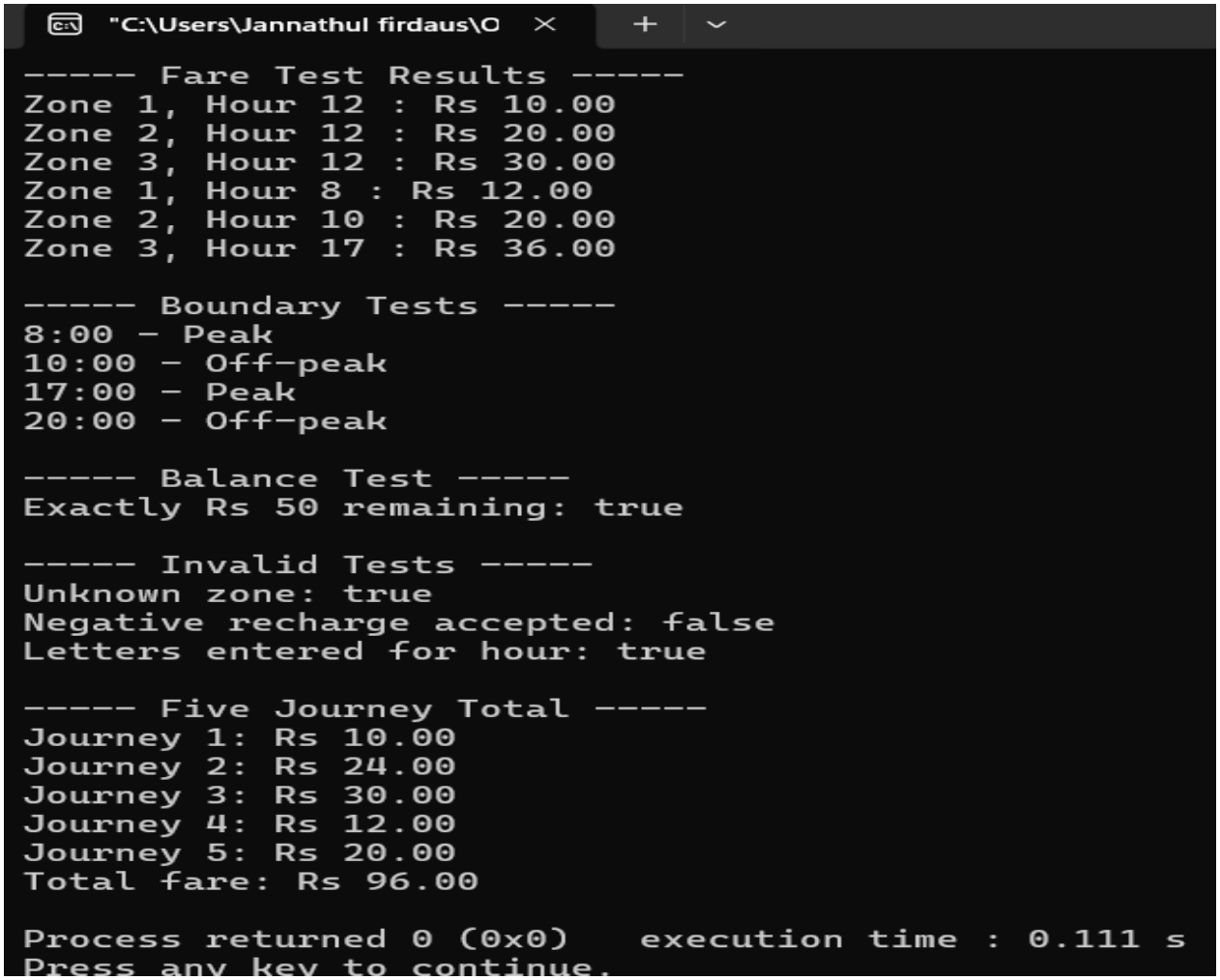
```
bool canTravel(long long balance, long long fare)
{
    return fare >= 0 && balance - fare >= MIN_BALANCE;
}
bool recharge(long long &balance, long long amount)
{
    if (amount < 0)
        return false;
    balance += amount;
    return true;
}
int main()
{
    cout << fixed << setprecision(2);
    cout << "----- Fare Test Results -----\n";
    int zones[] = {1, 2, 3, 1, 2, 3};
    int hours[] = {12, 12, 12, 8, 10, 17};
    for (int i = 0; i < 6; i++) {
        long long fare = calculateFare(zones[i], hours[i]);
        cout << "Zone " << zones[i]
             << ", Hour " << hours[i]
             << " : ";

        if (fare == -1)
            cout << "Invalid journey\n";
        else
            cout << "Rs " << fare / 100.0 << '\n';
    }
    cout << "\n----- Boundary Tests -----\n";
```

```
int boundaryHours[] = {8, 10, 17, 20};
for (int hour : boundaryHours) {
    cout << hour << ":00 - "
         << (isPeak(hour) ? "Peak" : "Off-peak") << '\n';
}
cout << "\n----- Balance Test -----\n";
cout << boolalpha
     << "Exactly Rs 50 remaining: "
     << canTravel(15000, 10000) << '\n';
cout << "\n----- Invalid Tests -----\n";
cout << "Unknown zone: "
     << (calculateFare(4, 12) == -1) << '\n';
long long balance = 10000;
cout << "Negative recharge accepted: "
     << recharge(balance, -500) << '\n';
string input = "abc";
istringstream inputStream(input);
int hour;
cout << "Letters entered for hour: "
     << !(inputStream >> hour) << '\n';
cout << "\n----- Five Journey Total -----\n";
int testZones[] = {1, 2, 3, 1, 2};
int testHours[] = {12, 8, 12, 17, 12};

long long total = 0;
for (int i = 0; i < 5; i++) {
    long long fare = calculateFare(testZones[i], testHours[i]);
    total += fare;
    cout << "Journey " << i + 1
```

```
        << ": Rs " << fare / 100.0 << '\n';
    }
    cout << "Total fare: Rs " << total / 100.0 << '\n';
    return 0;
}
```



2. Test table – Expected and actual results

Test case	Input	Expected result	Actual result
Inner off-peak	Zone 1, 12:00	₹10.00	₹10.00
Middle off-peak	Zone 2, 12:00	₹20.00	₹20.00
Outer off-peak	Zone 3, 12:00	₹30.00	₹30.00

Inner peak	Zone 1, 8:00	₹12.00	₹12.00
Middle off-peak	Zone 2, 10:00	₹20.00	₹20.00
Outer peak	Zone 3, 17:00	₹36.00	₹36.00
Boundary 20:00	Any valid zone	Off-peak	Off-peak
Exactly ₹50 remaining	Balance ₹150, fare ₹100	Accepted	true
Unknown zone	Zone 4	Invalid journey	Invalid journey
Negative recharge	-₹5	Rejected	false
Letters for hour	abc	Invalid input	true (input failure detected)

3. Five journeys – Hand calculation

Journey	**Zone**	**Time**	**Fare**
1	Inner	12:00	₹10.00
2	Middle	8:00	₹24.00
3	Outer	12:00	₹30.00
4	Inner	17:00	₹12.00
5	Middle	12:00	₹20.00
Total			₹96.00

4. Alternative concept and complexity analysis

Instead of writing separate test conditions, test cases can be stored in a table and processed using a loop.

```
struct TestCase {
    int zone;
    int hour;
    long long expectedFare;
};
TestCase tests[] = {
    {1, 12, 1000},
    {2, 8, 2400},
    {3, 12, 3000},
```

```
    {1, 17, 1200},
    {2, 12, 2000}
};
for (const TestCase& t : tests) {
    long long actual = calculateFare(t.zone, t.hour);


    cout << "Expected: " << t.expectedFare
         << ", Actual: " << actual << '\n';
}
```

Factor	Analysis
Time complexity	O(t), where t is the number of test cases
Space complexity	O(t) for storing the test table
Adding a boundary case	Add another entry to the test table
Advantage	Tests are organized and easier to maintain

TASK 10 – Compare Expectations with Execution (4 Marks)

1. Comparing double and integer-paise totals

For this experiment, a fixed fare of ₹10.00 (1,000 paise) per journey is used, based on the Inner Zone off-peak fare from Task 9.

The totals are calculated for 20,000, 500,000 and 3,000,000 journeys.

Number of journeys	Integer-paise total	Double total	Difference
20,000	₹2,00,000.00	₹2,00,000.00	₹0.00
500,000	₹50,00,000.00	₹50,00,000.00	₹0.00

3,000,000	₹3,00,00,000.00	₹3,00,00,000.00	₹0.00

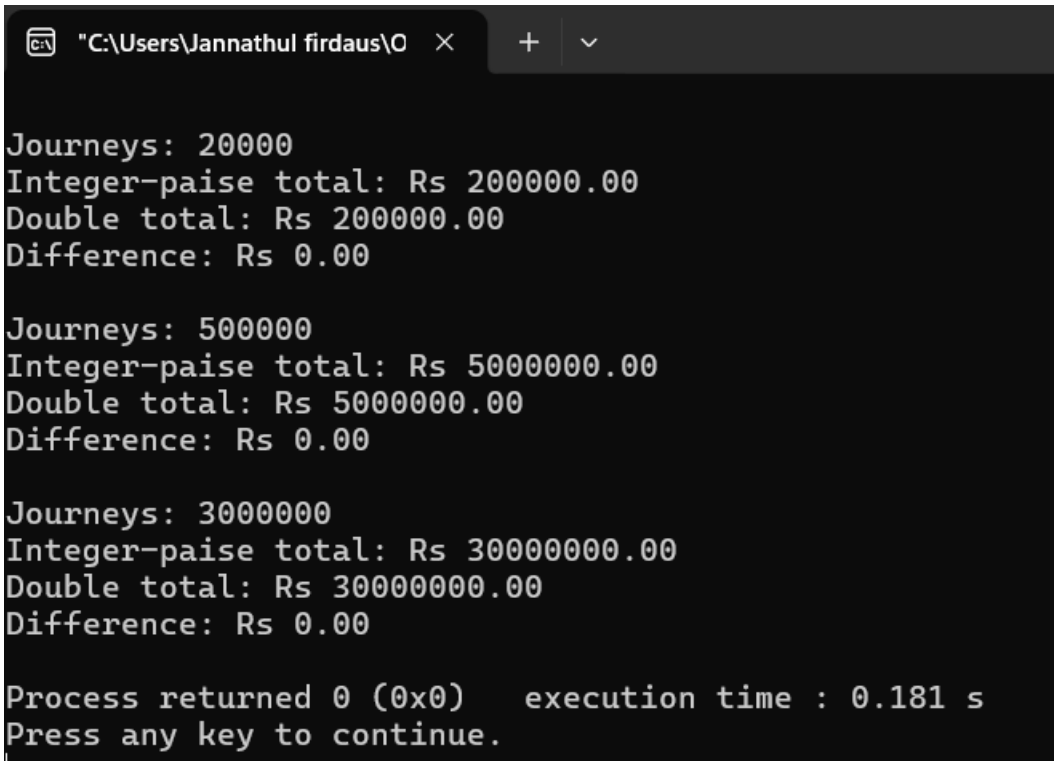
Observation: Both totals agree because ₹10.00 can be represented exactly as a binary floating-point value. Integer paise also stores the total exactly.

2. C++ program to compare the totals

```
#include <iostream>
#include <iomanip>
using namespace std;
int main()
{
   const int FARE_PAISE = 1000;
   const double FARE_RUPEES = 10.00;
   int journeys[] = {20000, 500000, 3000000};
   for (int n : journeys) {
      long long integerTotal = 0;
      double doubleTotal = 0.0;
      for (int i = 0; i < n; i++) {
         integerTotal += FARE_PAISE;
         doubleTotal += FARE_RUPEES;
      }
      cout << "\nJourneys: " << n << '\n';
      cout << "Integer-paise total: Rs "
           << fixed << setprecision(2)
           << integerTotal / 100.0 << '\n';

      cout << "Double total: Rs "
           << doubleTotal << '\n';
      cout << "Difference: Rs "
           << doubleTotal - integerTotal / 100.0 << '\n';
```

```
    }
    return 0;
}
```



3. Does int overflow for the network total?

A typical 32-bit int has a maximum value of 2,147,483,647.

Number of journeys	Total in paise	int result
20,000	20,000,000	No overflow
500,000	500,000,000	No overflow
3,000,000	3,000,000,000	Overflow

At 3,000,000 journeys, the total exceeds the maximum int value. Signed integer overflow in C++ causes undefined behaviour.

Therefore, long long should be used for the network total.

4. Compiler warnings

Compile the program using:

g++ -Wall -Wextra main.cpp -o main

- -Wall: Enables common compiler warnings.
- -Wextra: Enables additional compiler warnings.

- These options help identify possible programming mistakes.

no warnings appear:

"No warnings were reported by the compiler using -Wall -Wextra."

5. Reasons for differences

1. Integer paise stores monetary values as whole numbers.
2. Double uses binary floating-point representation.
3. Some decimal values cannot be represented exactly in binary.
4. Repeated floating-point additions may produce small rounding errors.
5. In this experiment, ₹10.00 is exactly representable, so both totals agree.

6. Alternative concept and complexity analysis

If the network total is accumulated using different data types:

Data type	**Behaviour at 3,000,000 journeys**	**Time per addition**	**Space**
int	Overflows because 3,000,000,000 exceeds its typical maximum	O(1)	Typically 4 bytes
long long	Stores the total exactly	O(1)	Typically 8 bytes
double	Stores the total approximately; ₹10.00 is exact in this experiment	O(1)	Typically 8 bytes

TASK 11 – Recommend and Defend the Final Design (4 Marks)

1. Recommended Final Design

I recommend using an object-oriented C++ design for the metro smart-card system.

- Types: Use int for zone numbers and hours, and long long for balance and fare values stored in paise. This avoids floating-point rounding errors and provides a large range for monetary calculations.
- Constants: Use constexpr or const for the minimum balance (₹50), base fares and peak surcharge (20%). This prevents accidental changes to important rules.
- Classes: Use a SmartCard class with private balance data and public functions such as canTravel(), deduct() and recharge(). This protects the balance from unauthorised changes.

2. How a Fare Revision Will Be Carried Out

If the metro changes its fares, the base fare values and surcharge constants can be updated in the program. The program must then be recompiled, tested and redeployed to the gates.

For example, if the Inner Zone fare changes from ₹10 to ₹12, update its fare value from 1000 to 1200 paise.

3. Limitations of the Design

- Fares are fixed in the compiled program.
- Every fare revision requires recompilation and redeployment.
- The current design supports only three zones.
- It does not include advanced features such as multiple routes, concessions or different passenger categories.

4. Correctness for the Whole Network

The design correctly calculates fares for the three defined zones and applies the 20% peak surcharge. My tests checked normal fares, peak-hour boundaries, invalid zones, negative recharge and minimum-balance conditions. The integer-paise calculation also avoids rounding errors in monetary totals.

However, the design must be extended and tested before being used across a larger metro network with additional zones and fare rules.

5. Alternative: Reading Fares from a Configuration File

Instead of compiling fares into the program, the gate can read the fare table from a configuration file during start-up.

Factor	**Compiled fare table**	**Configuration file**
Fare revision	Requires recompilation and redeployment	Change the file; no recompilation
Start-up time	No file-reading time	Reads z fare values
Time complexity	O(1) fare lookup	O(z) to load fares; O(1) lookup using an array
Extra space	O(z) for the fare array	O(z) for the loaded fare array
Maintenance	Less flexible	Easier to update

Here, z represents the number of zones. Reading the file takes O(z) time, and storing the fares requires O(z) space. For a small number of zones, the additional start-up cost is very low.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The new requirements affect the original design in the following ways:

Original component	Required change
Zone constants	Add Airport Zone with fare ₹60 (6000 paise).
Zone enum	Add Airport = 4.
Fare table	Increase from three entries to four.
Switch statement	Add a case for Airport Zone, or replace the switch with an indexed fare table.
Fare calculation	Add a 10% weekend discount.
Peak surcharge	Apply the 20% peak surcharge only on weekdays; do not combine it with the weekend discount.
SmartCard class	Modify it to support monthly-pass validation.
New MonthlyPass class	Store two selected zones and provide unlimited travel in those zones while the pass is active.

The original design cannot remain unchanged because it does not support the fourth zone, weekend discounts or monthly passes.

B. Evaluation of the Original Design (2 Marks)

1. Switch-based fare logic

The original switch statement can be modified by adding an Airport case. However, as more zones are added, maintaining multiple switch cases becomes difficult. An array-based fare table is easier to extend.

2. SmartCard class

The original SmartCard class can still manage balance, recharge and fare deduction. However, it needs to recognise monthly-pass journeys so that passengers with valid passes are not charged for covered zones.

Worked examples

Journey	Calculation	Final fare
Airport, weekday off-peak	Base fare	₹60
Airport, weekday peak	₹60 + 20%	₹72

Airport, weekend peak	₹60 − 10%	₹54
Middle, weekend off-peak	₹20 − 10%	₹18
Inner, valid monthly pass	Unlimited covered journey	₹0

The weekend discount replaces the peak surcharge. Both discounts are never applied together.

C. Adaptation or Redesign (2 Marks)

I recommend modifying the original design by retaining the SmartCard class and introducing an indexed fare table, separate fare rules and a MonthlyPass class.

Revised C++ implementation (C++17):

```
#include <iostream>
#include <array>
#include <iomanip>
using namespace std;
enum class Zone {
   Inner = 1,
   Middle,
   Outer,
   Airport
};
constexpr long long MIN_BALANCE = 5000;
constexpr int PEAK_RATE = 20;
constexpr int WEEKEND_DISCOUNT = 10;
const array<long long, 4> FARES = {
   1000, 2000, 3000, 6000
};
// Monthly pass for two selected zones
class MonthlyPass {
```

```
private:
    Zone zone1, zone2;
    bool active;
public:
    MonthlyPass(Zone z1, Zone z2)
        : zone1(z1), zone2(z2), active(true) {}
    bool covers(Zone z) const {
        return active && (z == zone1 || z == zone2);
    }
    void deactivate() {
        active = false;
    }
};
// Fare calculation
long long calculateFare(Zone zone, int hour, bool weekend) {
    int index = static_cast<int>(zone) - 1;
    if (index < 0 || index >= 4 ||
        hour < 0 || hour > 23) {
        return -1;
    }
    long long fare = FARES[index];
    // Weekend discount replaces peak surcharge
    if (weekend) {
        fare -= fare * WEEKEND_DISCOUNT / 100;
    }
    else if ((hour >= 8 && hour < 10) ||
             (hour >= 17 && hour < 20)) {
        fare += fare * PEAK_RATE / 100;
    }
```

```
    return fare;
}

class SmartCard {
private:
    long long balance;
public:
    SmartCard(long long amount) : balance(amount) {}
    bool travel(
        Zone zone, int hour, bool weekend,
        const MonthlyPass* pass = nullptr
    ) {
        if (pass != nullptr && pass->covers(zone)) {
            cout << "Monthly pass used. Fare: Rs 0.00\n";
            return true;
        }
        long long fare = calculateFare(zone, hour, weekend);
        if (fare < 0) {
            cout << "Invalid journey\n";
            return false;
        }

        if (balance - fare < MIN_BALANCE) {
            cout << "Insufficient balance\n";
            return false;
        }
        balance -= fare;
        cout << "Fare: Rs " << fare / 100.0
             << ", Balance: Rs " << balance / 100.0
```

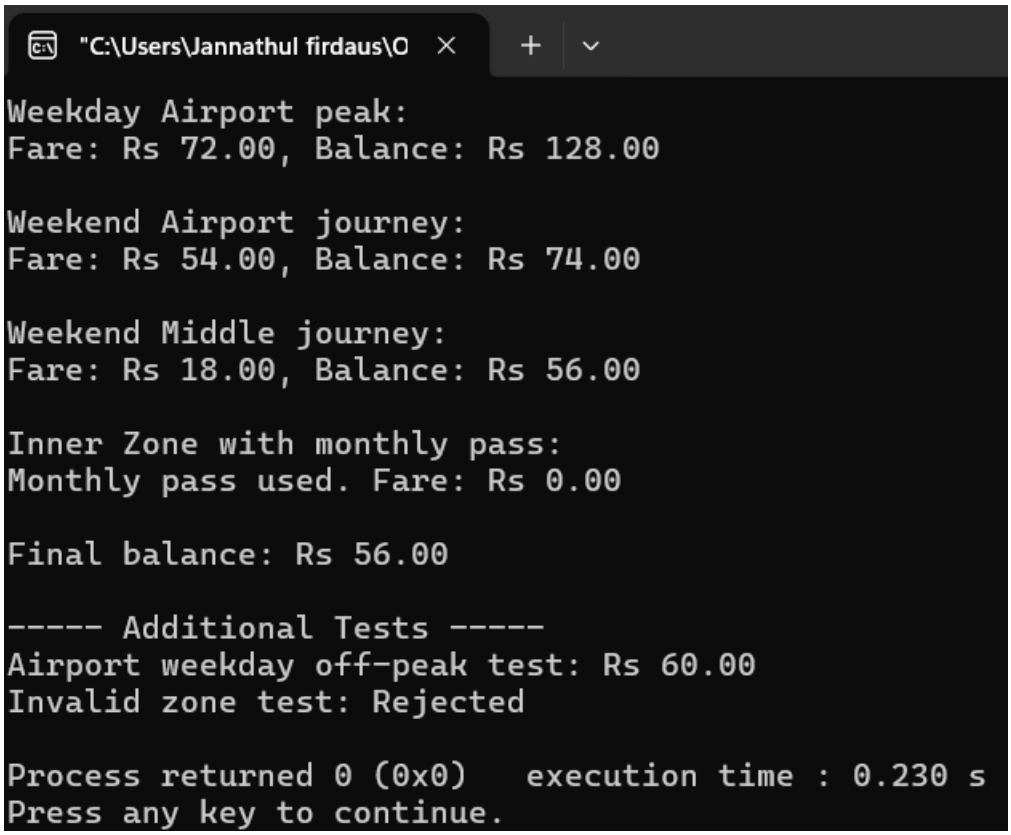
```
            << '\n';

        return true;
    }
    long long getBalance() const {
        return balance;
    }
};
int main() {
    cout << fixed << setprecision(2);
    SmartCard card(20000);
    MonthlyPass pass(Zone::Inner, Zone::Airport);
    cout << "Weekday Airport peak:\n";
    card.travel(Zone::Airport, 18, false);
    cout << "\nWeekend Airport journey:\n";
    card.travel(Zone::Airport, 18, true);
    cout << "\nWeekend Middle journey:\n";
    card.travel(Zone::Middle, 12, true);
    cout << "\nInner Zone with monthly pass:\n";
    card.travel(Zone::Inner, 12, false, &pass);
    cout << "\nFinal balance: Rs "
         << card.getBalance() / 100.0 << '\n';
    // Additional validation tests
    cout << "\n----- Additional Tests -----\n";
    long long offPeakFare =
        calculateFare(Zone::Airport, 12, false);
    cout << "Airport weekday off-peak test: Rs "
         << offPeakFare / 100.0 << '\n';
    long long invalidFare =
```

```
    calculateFare(static_cast<Zone>(5), 12, false);


  cout << "Invalid zone test: "
       << (invalidFare == -1 ? "Rejected" : "Accepted")
       << '\n';
  return 0;
}
```

Expected output:



D. Evidence-Based Defence (2 Marks)

The revised design was tested using the following cases:

Test case	**Expected result**	**Result**
Airport weekday off-peak	₹60	₹60
Airport weekday peak	₹72	₹72
Airport weekend peak	₹54	₹54
Middle weekend off-peak	₹18	₹18

Inner Zone with valid pass	₹0	₹0
Invalid zone	Rejected	Rejected by fare validation

Advantages:

- The fare array makes adding more zones easier.
- Integer paise maintains accurate monetary calculations.
- Separate fare rules prevent weekend discounts and peak surcharges from being combined.
- The MonthlyPass class separates pass-related rules from normal fare calculation.
- The SmartCard class continues to protect the passenger's balance.

Limitations:

- The example uses an active/inactive pass flag and does not check actual monthly expiry dates.
- It does not store passenger details or pass purchase information.
- The system would need additional validation for real metro routes and network-wide pass usage.

Complexity: Fare lookup takes O(1) time using the array. Checking the two pass zones also takes O(1) time. The fare table requires O(z) space for z zones.

Final decision: The original design should be modified, not discarded. The revised design supports the fourth zone, separate weekend and peak pricing rules, and unlimited journeys in two selected zones through a monthly pass. The test results demonstrate that the updated fare rules produce the expected values while preserving the original balance-protection mechanism.